Prakash Thapa
D20A- 32

Experiment No. 7: Introduction to Truffle, Establishing a Local Blockchain Using Ganache and Deployment of Smart Contract

Aim

To implement the blockchain platform Ganache, establish a local Ethereum blockchain network, and deploy and interact with a Solidity smart contract using MetaMask and Remix IDE.

Theory

1. Introduction to Ganache

Ganache is a personal Ethereum blockchain used for smart contract development, testing, and deployment. It provides a local blockchain environment where developers can simulate Ethereum transactions without using the actual Ethereum network.

It generates multiple pre-funded accounts with test Ether, allowing users to deploy smart contracts and perform transactions without spending real cryptocurrency.

Ganache provides the following features:

- Creates a local Ethereum blockchain for development and testing.
- Provides pre-funded accounts with test Ether.
- Supports instant mining of transactions.
- Displays transaction history, block details, and account balances.
- Allows developers to test smart contracts in a controlled environment.

2. Introduction to MetaMask

MetaMask is a cryptocurrency wallet that allows users to interact with Ethereum-based blockchain networks and decentralized applications (DApps). It manages blockchain accounts, stores private keys, and allows users to approve transactions.

In this experiment, MetaMask is connected to Ganache by configuring its local RPC network and importing an account using its private key.

3. Introduction to Remix IDE

Remix IDE is an online development environment used to write, compile, deploy, and interact with Solidity smart contracts. It provides tools for compiling Solidity programs and deploying them to different blockchain networks.

By connecting Remix IDE with MetaMask, smart contracts can be deployed using accounts available in the Ganache environment.

Tasks Performed

1. Installation and Setup of Ganache

- Downloaded and installed Ganache on the local system.
- Launched Ganache and created a new workspace.
- Started a local Ethereum blockchain network.
- Observed the generated accounts, account addresses, private keys, and available test Ether.

2. Connecting Ganache Accounts with MetaMask
- Opened the MetaMask browser extension.
- Added a custom network using the RPC URL provided by Ganache (http://127.0.0.1:7545).
- Configured the network details, including the network name, RPC URL, and Chain ID.
- Imported a Ganache account into MetaMask using its private key.
- Verified the account balance displayed in MetaMask.

3. Connecting Remix IDE with MetaMask
- Opened Remix IDE in the web browser.
- Created a new Solidity file.
- Navigated to the Deploy & Run Transactions section.
- Selected the Injected Provider - MetaMask environment.
- Connected MetaMask to Remix and verified the selected Ganache account.

4. Creating a Solidity Smart Contract
A simple storage smart contract was created to demonstrate how data can be stored and retrieved using blockchain transactions.

5. Compiling and Deploying the Smart Contract
- Selected the appropriate Solidity compiler version.
- Compiled the smart contract and checked for compilation errors.
- Selected the Injected Provider - MetaMask environment.
- Clicked on Deploy and approved the transaction through MetaMask.
- Successfully deployed the smart contract on the local Ganache blockchain.

6. Verifying Transaction Details in Ganache
- Opened the Ganache interface.
- Checked the transaction history to verify the deployment transaction.
- Observed the transaction hash, block number, gas usage, and account balance changes.
- Verified that the smart contract was deployed successfully.

7. Interacting with the Smart Contract
- Accessed the deployed contract through Remix IDE.
- Used the set() function to store a value in the smart contract.

- Used the automatically generated data getter function to retrieve the stored value.
- Verified that the updated value was reflected correctly.
- Observed the corresponding transaction details in Ganache.

Program

Solidity Smart Contract: SimpleStorage.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SimpleStorage {

   uint public data;

   function set(uint x) public {
      data = x;
   }
}
```

Program Explanation:

1. pragma solidity ^0.8.0; specifies the Solidity compiler version.
2. contract SimpleStorage defines a smart contract named SimpleStorage.
3. uint public data; declares a public unsigned integer variable that stores a value.
4. function set(uint x) public defines a function that accepts an unsigned integer as input.
5. data = x; updates the stored value in the blockchain.
6. Since data is public, Solidity automatically generates a getter function to retrieve its value.

Output

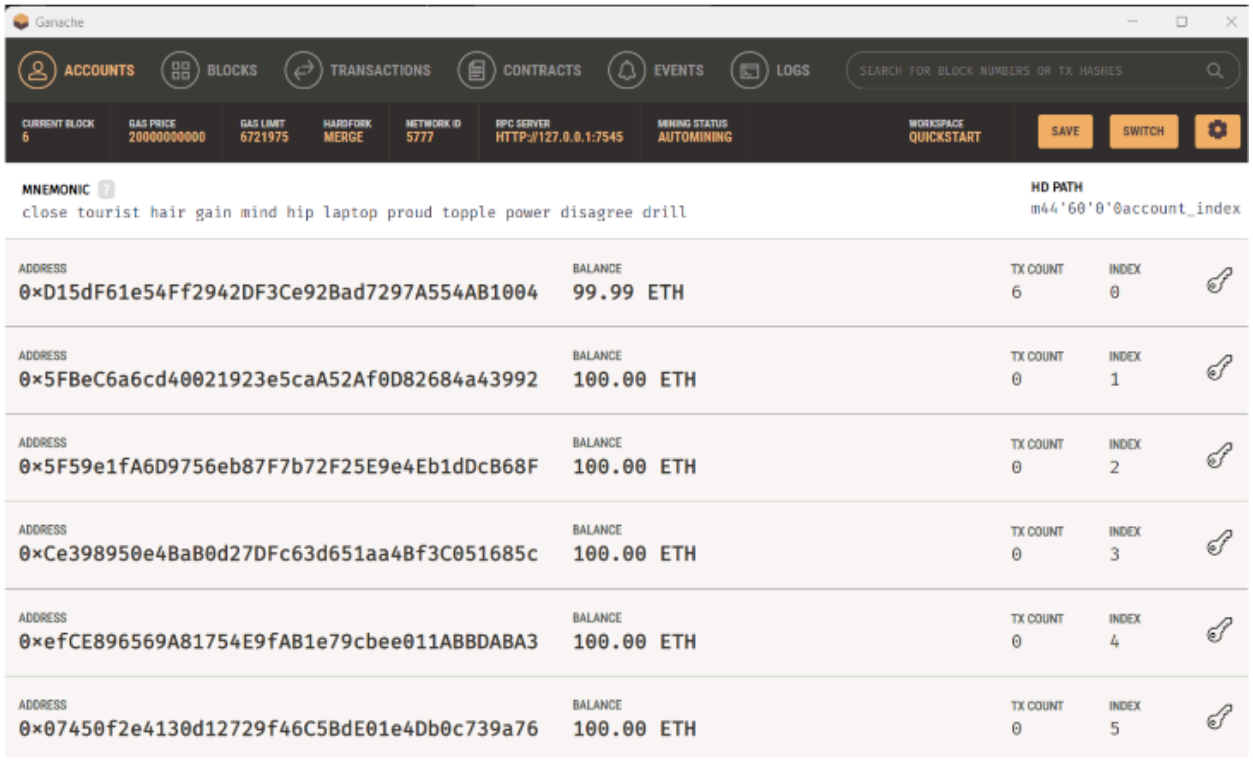
The following outputs were observed during the experiment:

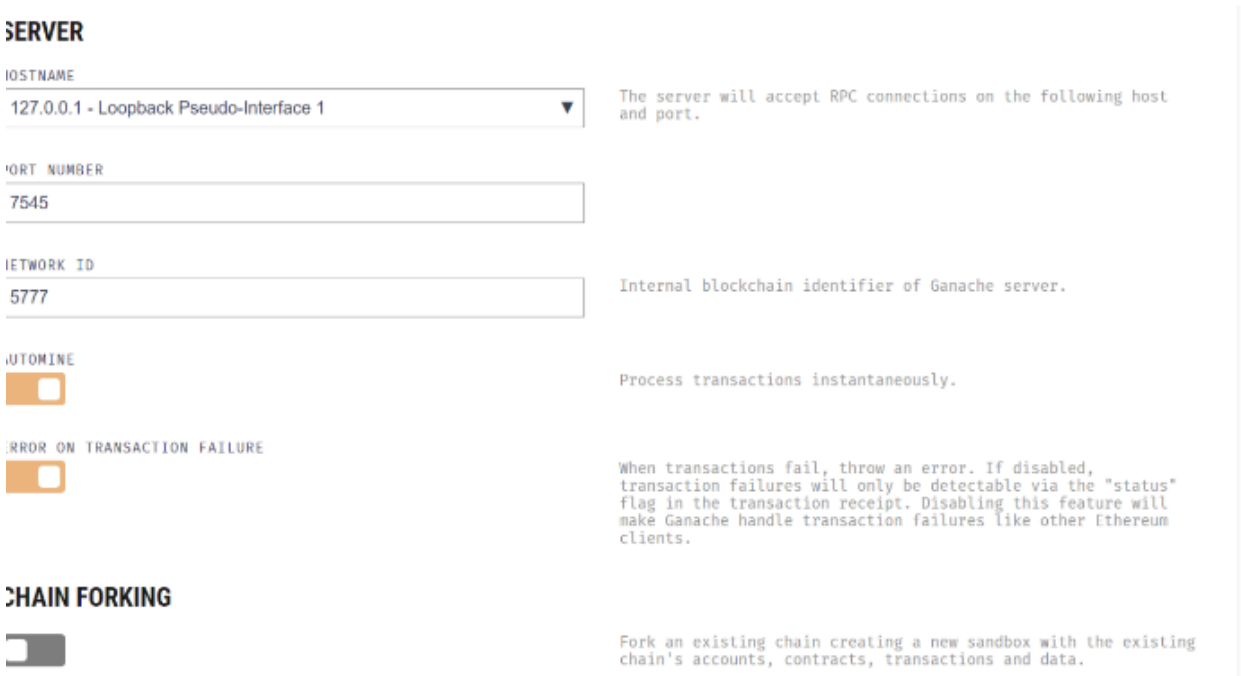
1. Ganache successfully established a local Ethereum blockchain with multiple pre-funded accounts.
2. MetaMask was successfully connected to the Ganache network.
3. Remix IDE was connected to MetaMask using the Injected Provider environment.
4. The SimpleStorage smart contract was successfully compiled and deployed.
5. The deployment transaction was recorded in the Ganache transaction history.
6. The set() function successfully stored the entered value in the smart contract.
7. The stored value was retrieved using the public variable getter function.

8. Account balances and transaction details were verified through Ganache.

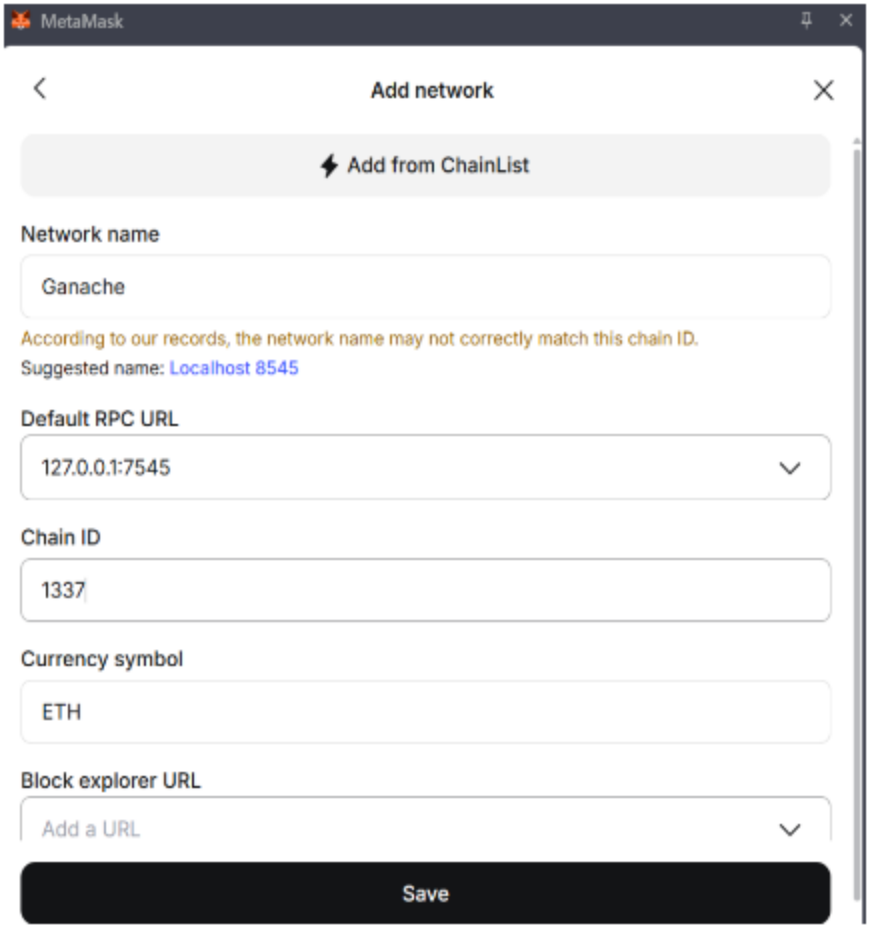
Screenshots:

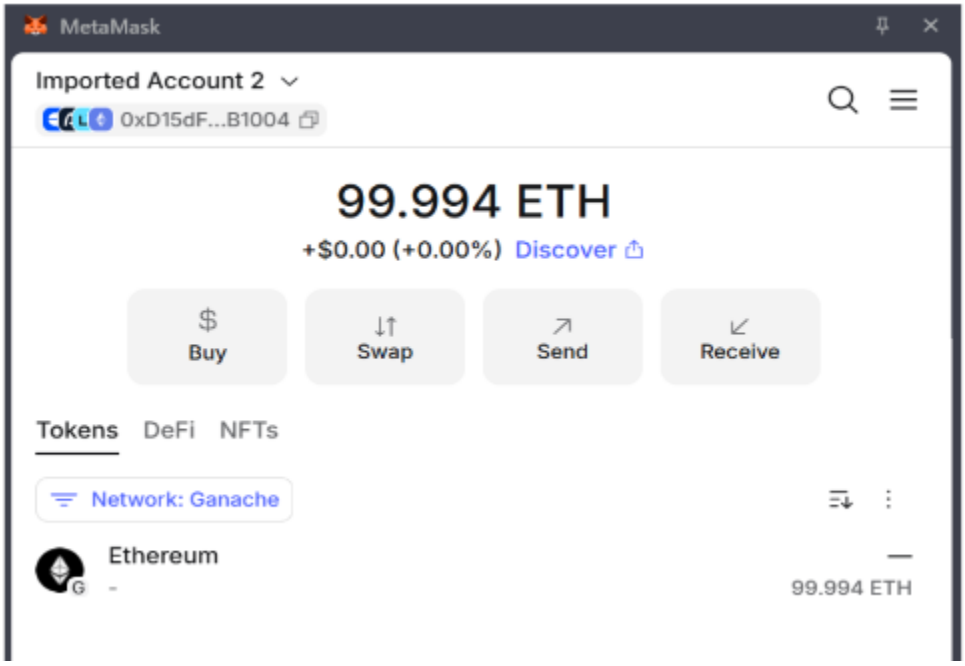
- Ganache workspace showing accounts and balances.



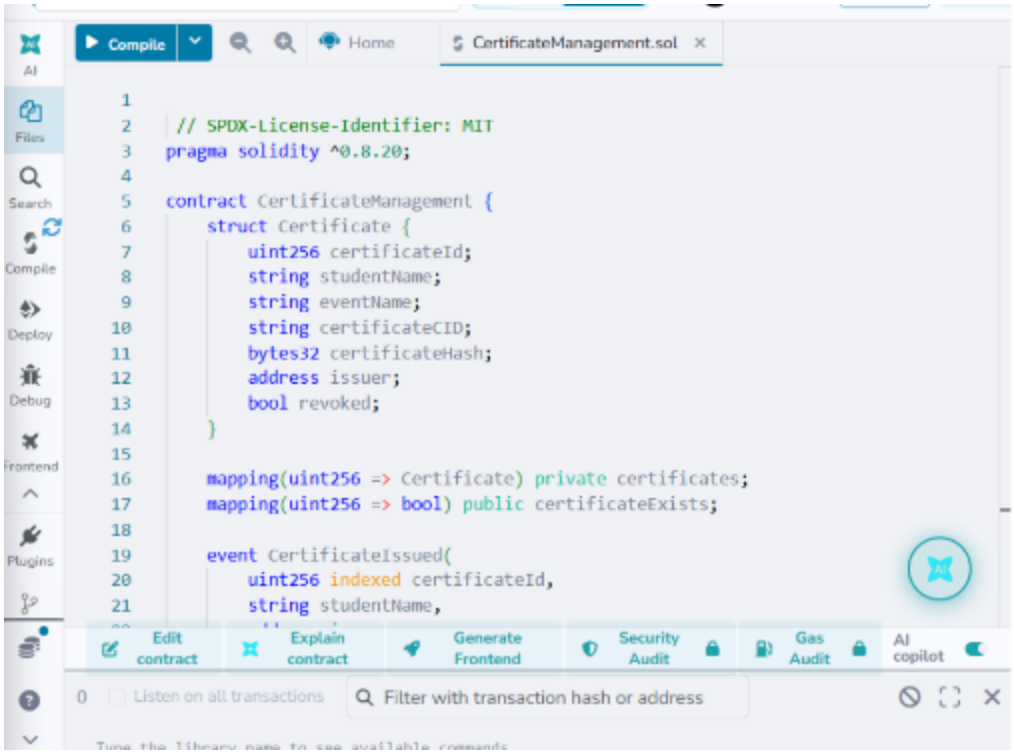


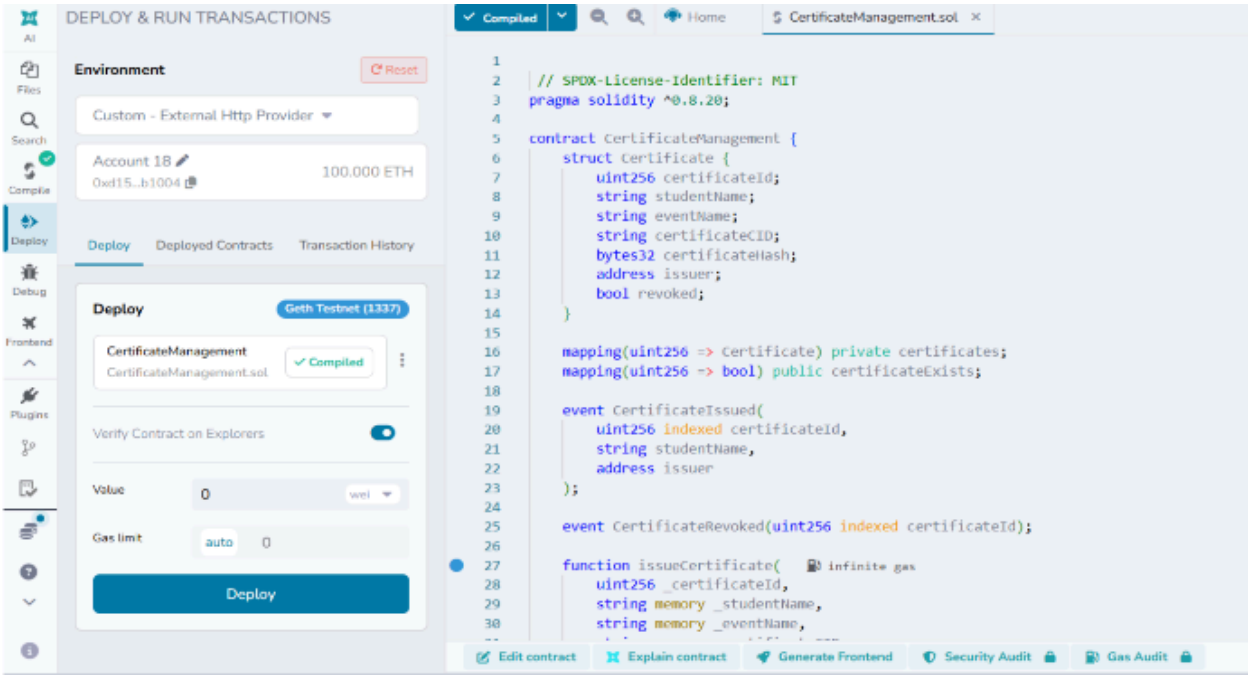
- MetaMask connected to the Ganache network.



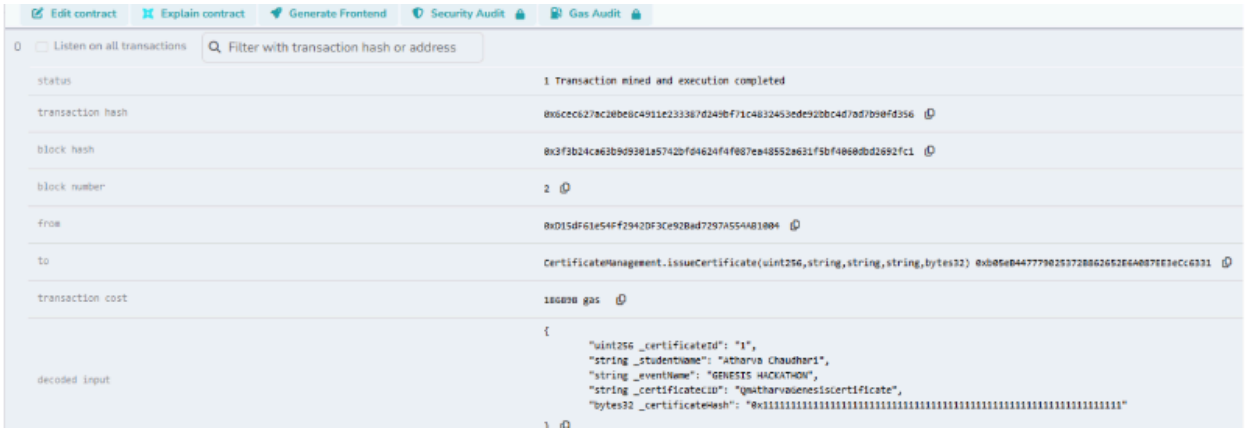


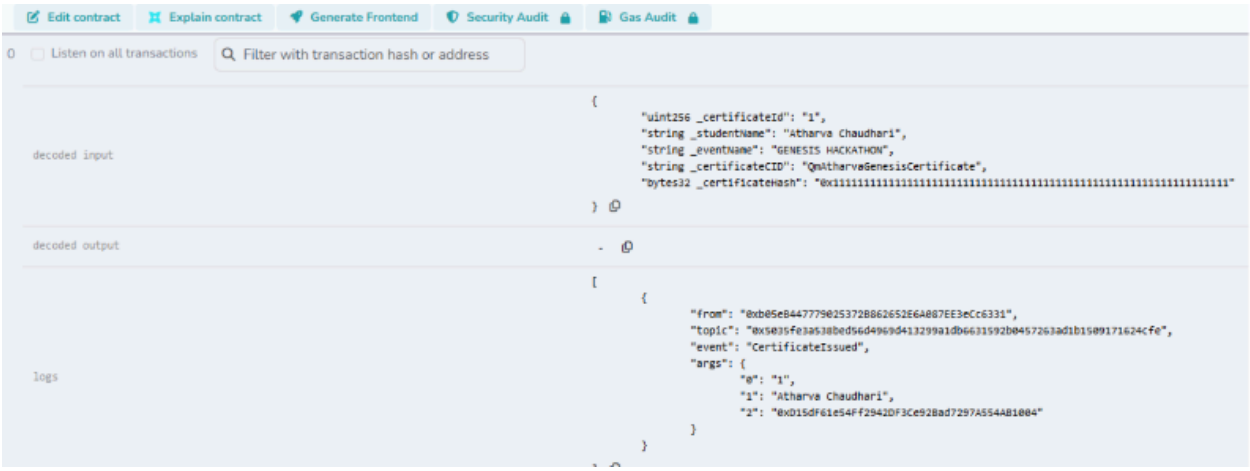
- Remix IDE showing the compiled smart contract.



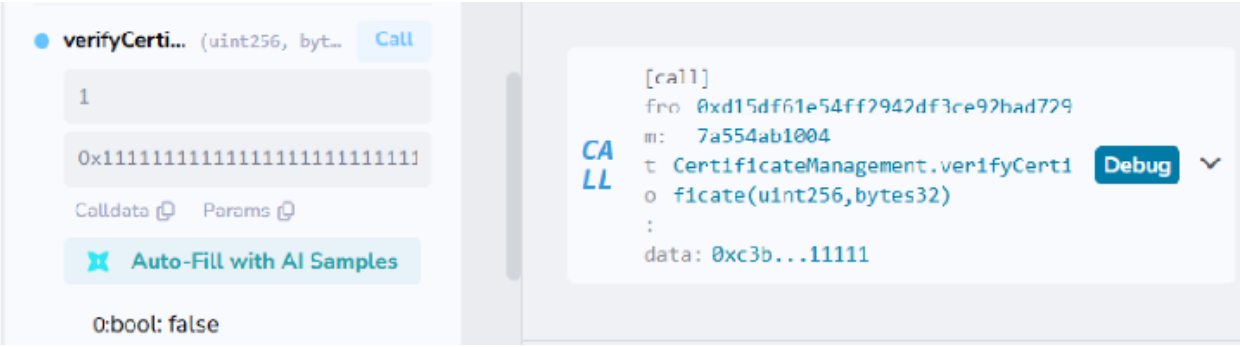


- MetaMask transaction confirmation during deployment.

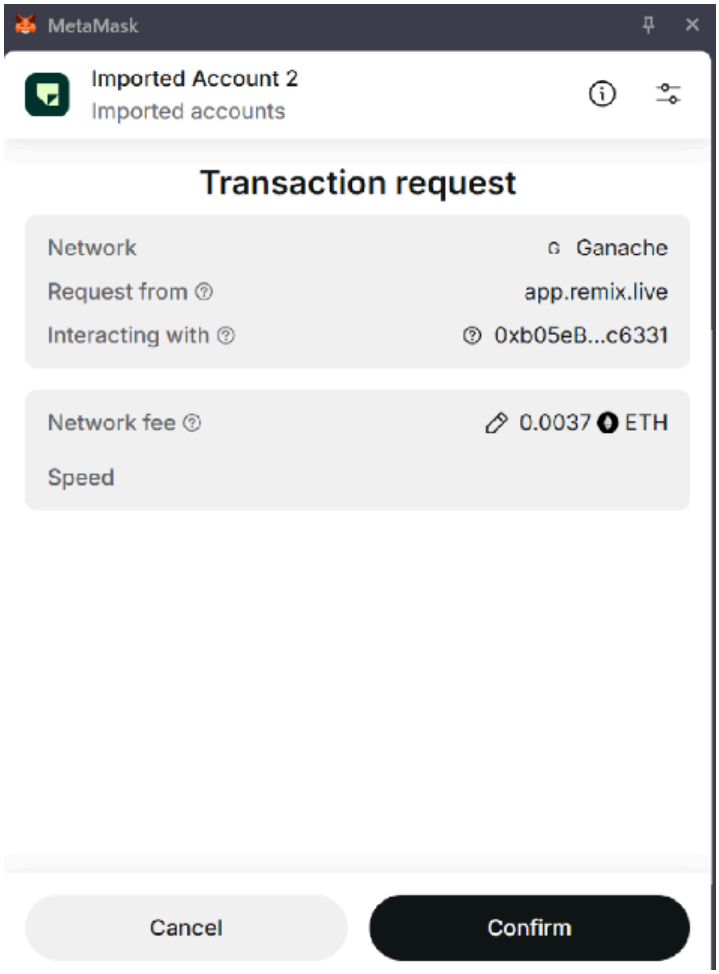




- Successfully deployed smart contract in Remix IDE.



- Ganache transaction history.




```
C:\Users\ADMIN\Desktop>mkdir Truffle_Exp7

C:\Users\ADMIN\Desktop>cd Truffle_Exp7

C:\Users\ADMIN\Desktop\Truffle_Exp7>truffle init

Starting init...
================

> Copying project files to C:\Users\ADMIN\Desktop\Truffle_Exp7

Init successful, sweet!

Try our scaffold commands to get started:
  $ truffle create contract YourContractName # scaffold a contract
  $ truffle create test YourTestName         # scaffold a test

http://trufflesuite.com/docs


C:\Users\ADMIN\Desktop\Truffle_Exp7>
```

```
Compiling your contracts...
===========================
> Compiling .\contracts\SimpleStorage.sol
> Artifacts written to C:\Users\ADMIN\Desktop\Truffle_Exp7\build\contracts
> Compiled successfully using:
   - solc: 0.8.21+commit.d9974bed.Emscripten.clang


Starting migrations...
======================
> Network name:    'development'
> Network id:      5777
> Block gas limit: 6721975 (0x6691b7)


1791921237_deploy_simple_storage.js
===================================

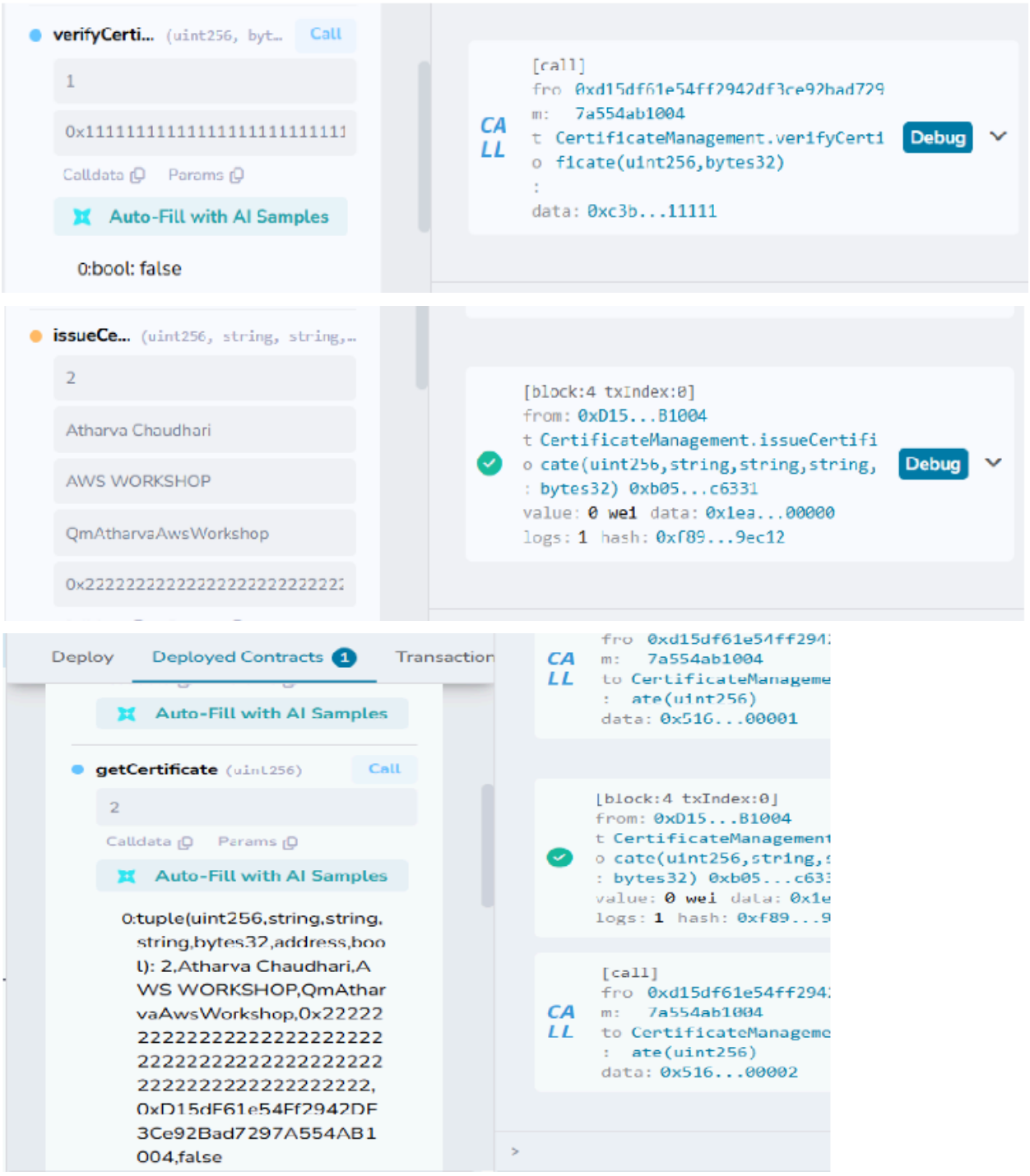
   Deploying 'SimpleStorage'
   -------------------------
   > transaction hash:    0x816f51017995f2b1330713b2a3e16eb9953a15b283c5470a733b552d1881
ce
   > Blocks: 0            Seconds: 0
   > contract address:    0xee663905897A510DbD6271155BA0E3412B2d85d2
   > block number:        7
   > block timestamp:     1791922120
   > account:             0xD15dF61e54Ff2942DF3Ce92Bad7297A554AB1004
   > balance:             99.994165447061248909
   > gas used:            125653 (0x1ead5)
   > gas price:           2.923977056 gwei
   > value sent:          0 ETH
   > total cost:          0.000367406489017568 ETH

   > Saving artifacts
   -------------------------------------
   > Total cost:     0.000367406489017568 ETH

Summary
=======
> Total deployments:   1
```

```
truffle(development)> let instance = await SimpleStorage.deployed()
undefined
truffle(development)> await instance.set(100)
{
  tx: '0x0c00b02f97e124f5c6e87e25d23dbdc21f6e90fdcafd04c24fe0065c18861d69',
  receipt: {
    transactionHash: '0x0c00b02f97e124f5c6e87e25d23dbdc21f6e90fdcafd04c24fe0065c18861d69',
    transactionIndex: 0,
    blockNumber: 8,
    blockHash: '0xb249fa17da3fd4c9395b32c42f531c451fbf5f26506413689a691059bfb85602',
    from: '0xd15df61e54ff2942df3ce92bad7297a554ab1004',
    to: '0xee663905897a510dbd6271155ba0e3412b2d85d2',
    cumulativeGasUsed: 43702,
    gasUsed: 43702,
    contractAddress: null,
    logs: [],
    logsBloom: '0x00000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000',
    status: true,
    effectiveGasPrice: 2872961262,
    type: '0x2',
    rawLogs: []
  },
  logs: []
}
truffle(development)> await instance.get()
BN {
  negative: 0,
  words: [ 100, <1 empty item> ],
  length: 1,
  red: null
}
truffle(development)>
```

- Smart contract interaction showing the stored and retrieved value.



Conclusion

In this experiment, we successfully established a local Ethereum blockchain environment using Ganache and learned how to integrate it with MetaMask and Remix IDE. We created, compiled, and deployed a Solidity smart contract on the local blockchain using a Ganache account.